CIMAT

Homework 3: KL Divergence and Maximum Likelihood Estimation of the Graphic Model

Pedro Llerenas [1]

[1] CIMAT pedro.llerenas@cimat.mx

Abstract: We explore three models for two databases. One model considers independent random variables, another one is a chain of dependencies, and the last one is a dependency tree. We calculate their maximum likelihood and their pair-wise Kullback-Leibler divergence, where we observe that the dependency tree model give a better fit to the generated databases. The chain of dependencies follows in fitting, and lastly and independent nodes model, which gives a poor representation of the joint probabilities in the model.

Keywords: Kullback-Leibler Divergence; Mutual Information; Normal Distribution; Maximum Likelihood; Graphic Model.

Introduction

Being able to predict an outcome of a certain event will remain to be the most powerful tool humanity possesses. To be able to do this, we attempt to describe these events with mathematical models. Although these models cannot accurately predict a future events with 100% certainty, being able to deduce that an outcome is *likely* to occur, allows judging the situations an easier task.

Once an event is has been sampled over many iterations, we can observe patterns in the data they generate. We use known probability distributions that may follow these patterns. To be able to justify why a certain distribution *fits* the observed data, we use maximum likelihood estimates, which tell us how likely it is to observe what was observed, if we assume a certain distribution is the source of the data. Maximizing this likelihood then leads to properly adjusted distributions.

In this document, we use Mutual-Information-Maximizing Input Clustering

Preliminaries

Before we present the models, we provide some definitions.

Entropy

For a discrete random variable X, we denote by $H(X)$ the *entropy* of X. We define it as

$$H(X) = -\sum_{x \in X} P(x) \log P(x).$$

This value quantifies the average level of uncertainty or information associated with the variable's potential states or possible outcomes.

Mutual Information

For a pair of random variables X, Y, the mutual information $I(X;Y)$ measures how much information about X is gained from knowing Y. It is defined as

Citation: Pedro Llerenas. Homework 3: KL Divergence and Maximum Likelihood Estimation of the Graphic Model. **2026,** 22, September.

$$I(X;Y) = \sum_{x \in X} \sum_{y \in Y} P(x,y) \log \frac{P(x \mid y)}{P(x)} = H(X) - H(X|Y) = H(Y) - H(Y|X)$$

Maximum Likelihood Estimation

This method estimates the parameters of an assumed probability distribution, given some observed data. We model a set of observations as a random sample from an unknown joint probability distribution which is expressed in terms of a set of parameters. We write the parameters governing the joint distribution as a vector $\theta = [\theta_1, ..., \theta_k]^\top$ so that this distribution falls within a parametric family $\{f(\cdot;\theta) \mid \theta \in \Theta\}$, where Θ is called the parameter space, a finite-dimensional subset of Euclidean space. Evaluating the joint density at the observed data sample $\boldsymbol{y} = (y_1, ..., y_n)$ gives a real-valued function,

$$\mathcal{L}_n(\theta) = \mathcal{L}_n(\theta; \boldsymbol{y}) = f_n(\boldsymbol{y}; \theta)$$

which is called the likelihood function. For independent random variables $f_n(\boldsymbol{y};\theta)$ will be the product of univariate density functions:

$$f_n(\boldsymbol{y};\theta) = \prod_{k=1}^{n} f_k^{\text{univar}}(y_k;\theta).$$

Consequently, the log-likelihood is given by

$$\log(f_n(\boldsymbol{y};\theta)) = \sum_{k=1}^{n} \log f_k^{\text{univar}}(y_k;\theta).$$

Models

Independent Variables

We first explore the simplest model possible: independent random variables. This produces a joint probability distribution

$$P(X) = \prod_{1 \le i \le n} P(X_i)$$

MIMIC

This algorithm was first proposed in [1], it takes a combinatorial approach to the best model. That is, it produces a first-order dependency chain (a simple Bayesian network) to represent the joint probability. It finds the optimal ordering $X_{\pi(1)}, X_{\pi(2)}, ..., X_{\pi(n)}$ by minimizing the Kullback-Leibler divergence between the true distribution of the elite solutions and the estimated chain distribution. Mathematically, this reduces to finding the chain that maximizes the mutual information between adjacent variables.

Algorithm 1: MIMIC

$X :=$ Random initial population of size N
Evaluate fitness $f(x)$ for all $x \in X$
while not converged **do**
  $\theta_t :=$ Fitness threshold for top θ-percentile
  $E := \{x \in X \mid f(x) \ge \theta_t\}$ (Elite subset)
  Compute marginal $H(X_i)$ and conditional $H(X_i \mid X_j)$ entropies for all pairs in E
  Find permutation π to minimize total conditional entropy: $H\left(X_{\pi(1)}\right) + \sum_{i=1}^{n-1} H\left(X_{\pi(i+1)} \mid X_{\pi(i)}\right)$
  $X :=$ Sample N new candidates from this dependency chain
  Evaluate fitness $f(x)$ for all new $x \in X$
end
return Best candidate $x^* \in X$

In this document, we will focus on $\theta = 100\%$. That is, without any elitism. The algorithm then simplifies to the following.

Algorithm 2: Simplified MIMIC

$X :=$ Random initial population of size N
Compute marginal $H(X_i)$ and conditional $H\left(X_i \mid X_j\right)$ entropies for all pairs.
Find permutation π to minimize total conditional entropy: $H\left(X_{\pi(1)}\right) + \sum_{i=1}^{n-1} H\left(X_{\pi(i+1)} \mid X_{\pi(i)}\right)$
return π

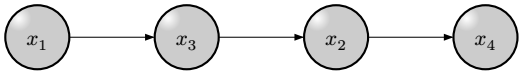


Figure 1: Example of a first-order dependency chain.

Dependency Tree

This model is similar to MIMIC in their goal of maximizing mutual information (or minimize entropy). However, dependency tree does not restrict the number of children a node can have, producing a directed acyclic graph, where each variable has at most one parent.

The following algorithm is also a simplified version of dependency tree, where we do not consider elitism. It was first presented in [2].

Algorithm 3: Dependency Tree

$X :=$ Random initial population of size N
Compute Mutual Information $I(X_i; X_j)$ for all pairs of variables
Let $G = (V, E)$ be a complete graph where weight $W(i, j) = I\left(X_i; X_j\right)$
$T :=$ Maximum Spanning Tree of G (via Kruskal)
Select an arbitrary node $R \in V$ as the root
Direct all edges in T outward from R to form a dependency tree
Estimate marginal $P(R)$ and conditionals $P\left(X_i \;\middle|\; X_{\text{parent}(i)}\right)$
return pointer to R (root of tree)

Experiment Setup

Experiment 1

We will use be using 6 binary random variables for which we have the following empirical data

Variable (X_i)	**Parent** (X_{i-1})	$P(X_i = 1 \mid X_{i-1} = 0)$	$P(X_i = 1 \mid X_{i-1} = 1)$
x_1	None	–	0.20
x_2	x_1	0.92	0.31
x_3	x_2	0.42	0.15
x_4	x_3	0.28	0.75
x_5	x_4	0.67	0.43
x_6	x_5	0.35	0.82

Table 1: Conditional probabilites for the first database DB1.



Figure 2: Dependency chain from DB1. Its joint probability is given by $P(X) = P(X_1) \prod_{i=2}^{6} P(X_i|P_{i-1})$

This data resembles a simple dependency chain, for which we expect MIMIC to perform well. Dependency tree should also perform well.

Experiment 2

The next experiment also contains 6 binary random variables.

Variable (X_i)	**Parent** (X_{parent})	$P(X_i = 1 \mid X_{\text{parent}} = 0)$	$P(X_i = 1 \mid X_{\text{parent}} = 1)$
x_1	None	–	0.20
x_2	x_1	0.92	0.31
x_4	x_1	0.42	0.15
x_5	x_2	0.28	0.75
x_6	x_2	0.67	0.43
x_3	x_6	0.35	0.82

Table 2: Conditional probabilities for the second database DB2.

In this case, x_1 contains 2 children, and so does x_2. We expect dependency tree to perform well, but MIMIC should not be able to reproduce the same dependency list (by design).

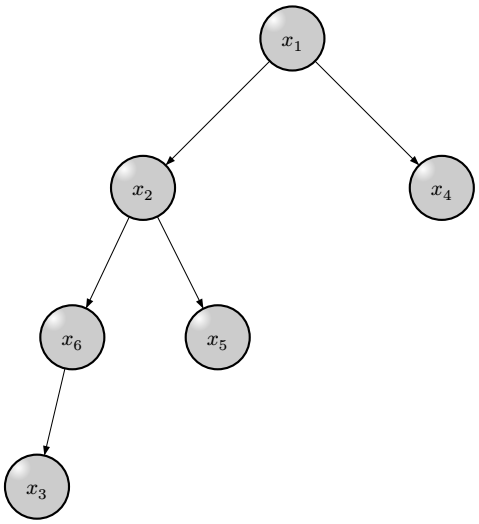


Figure 3: Dependency tree generated by DB2. Its joint probability is given by $P(X) = P(X_1)P(X_2|X_1)P(X_4|X_1)P(X_6|X_2)P(X_5|X_2)P(X_3|X_6)$

Results

Independent Variables (Model 1)

Maximum Likelihood Estimation

For this model, since we do not have dependencies, we only need to observe the marginal probabilities. For an execution with a database size of 30,000 we obtained the following probabilities, which also correspond to the maximum likelihood estimation.

Bit	Probability
$P(X_1 = 1)$	0.20198
$P(X_2 = 1)$	0.79633
$P(X_3 = 1)$	0.20297
$P(X_4 = 1)$	0.37415
$P(X_5 = 1)$	0.58213
$P(X_6 = 1)$	0.62222

Table 3: Values for Database 1.

Bit	Probability
$P(X_1 = 1)$	0.2023
$P(X_2 = 1)$	0.79675
$P(X_3 = 1)$	0.22539
$P(X_4 = 1)$	0.37619
$P(X_5 = 1)$	0.48014
$P(X_6 = 1)$	0.72331

Table 4: Values for Database 2.

Despite having identical probabilities a priori, the values are different, as we used the same number generator to first generate DB1, then DB2, without resetting it. This is the case throughout the whole program; we only use 1 instance of `std::mt19937`.

Kullback-Leibler Divergence

We obtained $D_{\mathrm{KL}}(M_1 \parallel \mathrm{DB1}) = 0.587202$, and $D_{\mathrm{KL}}(M_1 \parallel \mathrm{DB2}) = 0.587202$.

MIMIC (Model 2)

Resulting Chain

We produced the chain

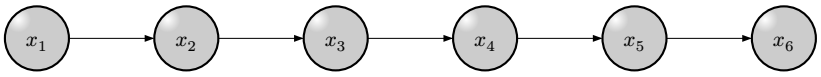


Figure 4: Chain generated by MIMIC for DB1.

Which corresponds to the original model used to generate the database. Note that sometimes (rarely, maybe $p < 0.05$), we obtain slightly different chains. This is due to the randomness of the generation, which can cause some weakly dependent nodes to lose their original pairing.

For database 2, the following chain was generated.

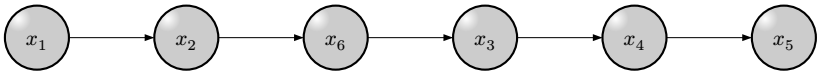


Figure 5: Chain generated by MIMIC for DB2.

It was expected to not even be close to the true nature of the database, as it was originally a tree, so we cannot expect a simple chain to approximate it well.

Maximum Likelihood Estimation

The produced joint probability values are presented in the following tables.

Bit	Probability
$P(x_1 = 1)$	0.20198
$P(x_2 = 1 \mid x_1 = 0)$	0.92123
$P(x_2 = 1 \mid x_1 = 1)$	0.302852
$P(x_3 = 1 \mid x_2 = 0)$	0.419158
$P(x_3 = 1 \mid x_2 = 1)$	0.147677
$P(x_4 = 1 \mid x_3 = 0)$	0.277769
$P(x_4 = 1 \mid x_3 = 1)$	0.752624
$P(x_5 = 1 \mid x_4 = 0)$	0.672158
$P(x_5 = 1 \mid x_4 = 1)$	0.431538
$P(x_6 = 1 \mid x_5 = 0)$	0.348194
$P(x_6 = 1 \mid x_5 = 1)$	0.818924

Table 5: Probability tables by MIMIC for DB1.

Bit	Probability
$P(x_1 = 1)$	0.2023
$P(x_2 = 1 \mid x_1 = 0)$	0.920133
$P(x_2 = 1 \mid x_1 = 1)$	0.310232
$P(x_6 = 1 \mid x_2 = 0)$	0.3477
$P(x_6 = 1 \mid x_2 = 1)$	0.819128
$P(x_3 = 1 \mid x_6 = 0)$	0.419567
$P(x_3 = 1 \mid x_6 = 1)$	0.151111
$P(x_4 = 1 \mid x_3 = 0)$	0.36643
$P(x_4 = 1 \mid x_3 = 1)$	0.409734
$P(x_5 = 1 \mid x_4 = 0)$	0.461791
$P(x_5 = 1 \mid x_4 = 1)$	0.510566

Table 6: Probability tables by MIMIC for DB2.

KL-Divergence

We obtained $D_{\mathrm{KL}}(M_2 \parallel \mathrm{DB1}) = 0.000453911$, and $D_{\mathrm{KL}}(M_2 \parallel \mathrm{DB2}) = 0.587202$.

Chow-Liu Tree (Model 3)

Tree Generated

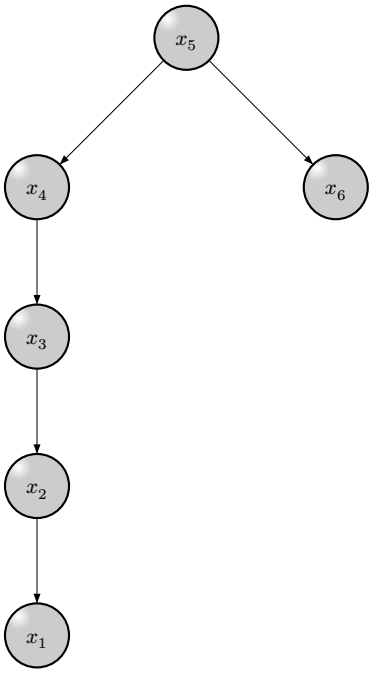


Figure 6: Dependency tree generated by the Chow-Liu Algorithm for DB1.

We observe that although it is not exactly the chain the database came from, this is expected, as taking an arbitrary root node leads to this. We can observe that if we instead take node 1 as the root, it leads to the original chain (by reversing the edges).

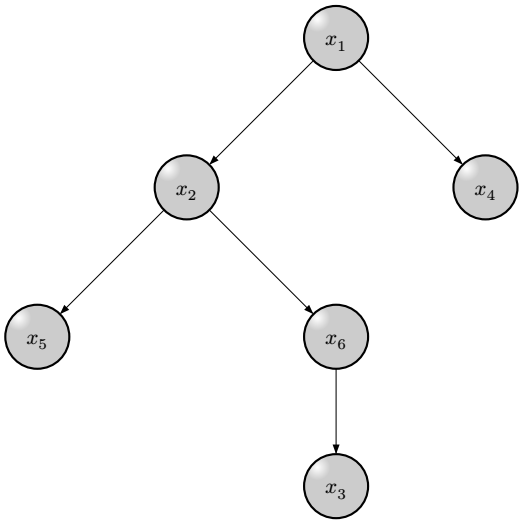


Figure 7: Dependency tree generated by the Chow-Liu Algorithm for DB2.

This tree is verbatim to the original tree. Since it is random, it is also possible to obtain other permutations of the tree, where the root may not be node 1. However, the joint probability stays the same.

Maximum Likelihood Estimate

Variable (X_i)	**Parent** (X_{parent})	$P(X_i = 1 \mid X_{\text{parent}} = 0)$	$P(X_i = 1 \mid X_{\text{parent}} = 1)$
x_5	None	–	0.58213
x_4	x_5	0.508986	0.277361
x_3	x_4	0.0802269	0.408285
x_2	x_3	0.851574	0.579396
x_1	x_2	0.691363	0.0768149
x_6	x_5	0.348194	0.818924

Table 7: Conditional probabilities for the second database DB1 generated by the Chow-Liu tree.

Variable (X_i)	**Parent** (X_{parent})	$P(X_i = 1 \mid X_{\text{parent}} = 0)$	$P(X_i = 1 \mid X_{\text{parent}} = 1)$
x_1	None	–	0.2023
x_2	x_1	0.920133	0.310232
x_5	x_2	0.672226	0.431139
x_6	x_2	0.3477	0.819128
x_3	x_6	0.419567	0.151111
x_4	x_1	0.280343	0.754128

Table 8: Conditional probabilities for the second database DB2 generated by the Chow-Liu tree.

KL-Divergence

We obtained $D_{\text{KL}}(M_3 \parallel \text{DB1}) = 0.000453911$, and $D_{\text{KL}}(M_3 \parallel \text{DB2}) = 0.000399334$.

Conclusions

We may conclude that Chow-Liu tree produces the best results for both databases in general. In particular, it does not suffer the 1-dimensionality problem of the MIMIC model, which only allows for 1 child node. The independent nodes model is extremely simple, but it does not capture the true nature of the databases, since it completely ignores the joint probabilities.

Ideas

One possible way of allowing the MIMIC model to have greater versatility is to choose 1 root node, and allow only the root node to have various children. This keeps an almost linear model, but may allow leeway for some generalizations.

Appendix

Compilation Instructions

Requirements:

- C++20
- CMake
- Make

First, `cd` into the directory. Then,

```
mkdir build && cd build
```

Then, we use CMake to generate the make files.

```
cmake ..
```

Finally, we use `make` to compile and obtain the necessary binary files.

```
make
```

Execution Instructions

All the results are obtained from a single execution, all printed to `cout`. Once all compilations are done, we do (while in the `build` directory)

```
./src/main
```

This will generate results for all 3 models, for both databases.

Bibliography

[1] J. S. D. Bonet, C. L. Isbell, and P. A. Viola, "MIMIC: Finding Optima by Estimating Probability Densities," in *Neural Information Processing Systems*, 1996. [Online]. Available: https://api.semanticscholar.org/CorpusID:6126501

[2] C. Chow and C. Liu, "Approximating discrete probability distributions with dependence trees," *IEEE Transactions on Information Theory*, vol. 14, no. 3, pp. 462–467, 1968, doi: 10.1109/TIT.1968.1054142.